



01211373 Machine Learning and Programming for Industry

# Regression Analysis

Week 3 — Classical Machine Learning

---

# Last Week → This Week

## WEEK 2 RECAP

- Four classifiers, one interface: fit / predict / score
- Trees skip scaling; logistic reg., k-NN, SVM all need it
- Missing data: drop, impute, or flag-and-fill
- ColumnTransformer + Pipeline: preprocess safely, every time

## TODAY

- Ordinary least squares & multiple linear regression
- Evaluating regression: MSE, RMSE,  $R^2$
- Overfitting, illustrated with polynomial regression
- Taming it: Ridge (L2) and Lasso (L1) regularization
- Random forest regression as a nonlinear alternative

# Today's Running Example: Sensor Calibration

Two regression problems we'll return to all lecture — and you'll build both in the lab.

## Force Sensor Calibration

A strain-gauge outputs a raw voltage. What applied force, in Newtons, does that voltage actually correspond to?

## Joint Friction Estimation

Given velocity, temperature, and load, predict the friction torque a joint controller needs to compensate for.

*Both outputs are real numbers, not categories — that's the whole difference from Week 2.*

# From Classification to Regression

WEEK 2

## Classification

*“Is this bearing reading Normal or Faulty?”*

Output: one of a fixed set of classes

WEEK 3

## Regression

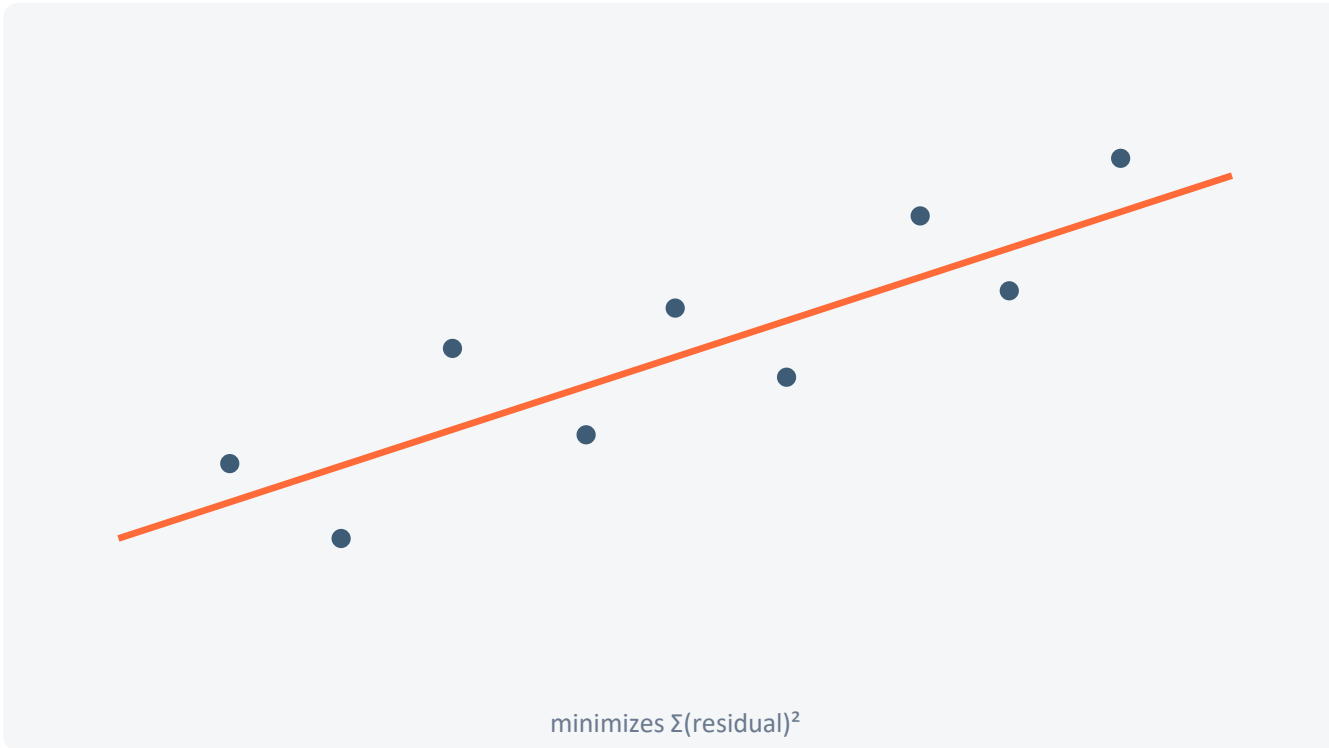
*“What force, in Newtons, does this voltage correspond to?”*

Output: a real-valued number

# Ordinary Least Squares



Finds the line that minimizes the sum of squared vertical distances between the line and each data point.



$$y = w_1x + w_0$$

Fast, interpretable, and a strong first baseline before reaching for anything fancier — same philosophy as logistic regression in Week 2.

```
from sklearn.linear_model import LinearRegression
```

# From One Feature to Many



The same idea extends directly to several input features — one weight per feature, still a single flat model.

$$\text{friction\_torque} = w_0 + w_1 \cdot \text{velocity} + w_2 \cdot \text{temperature} + w_3 \cdot \text{load}$$

**Each coefficient is directly interpretable:**  $w_1$  is “extra torque per unit of velocity, holding temperature and load fixed.” That's a real advantage over most classifiers from Week 2 — you can hand these numbers to a controls engineer.

# Evaluating Regression Models



Accuracy doesn't apply to a continuous output — these three metrics take its place.

## MSE

Mean squared error. Penalizes large errors heavily; same units as the target, squared.

## RMSE

$\sqrt{\text{MSE}}$ . Back in the target's original units — easiest to reason about directly (e.g., “off by 1.2 N”).

## $R^2$

Fraction of variance explained. 1.0 = perfect; 0.0 = no better than predicting the mean.

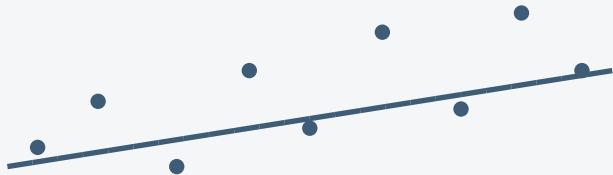
# The Overfitting Problem



A straight line can't follow a curve. But letting the model bend too freely just memorizes the noise instead.

## Underfit

degree 1



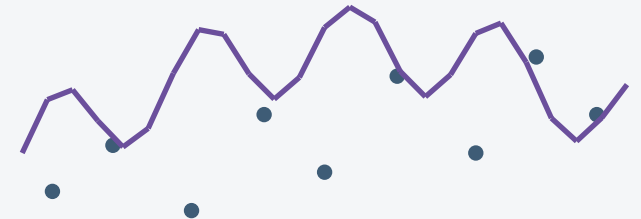
## Good Fit

degree 2



## Overfit

degree 10





# Regularization: Ridge (L2)



Adds a penalty on the sum of squared coefficients — the model is rewarded for fitting the data AND for keeping weights small.

$$\text{loss} = \text{MSE} + \alpha \cdot \sum w_i^2$$

**$\alpha$  controls the trade-off:**  $\alpha = 0$  is plain OLS; larger  $\alpha$  shrinks every coefficient toward zero (but rarely to exactly zero), reducing variance at the cost of a little bias. This is exactly what tames the degree-10 overfit curve from the last slide.

# Regularization: Lasso (L1)



Same idea, different penalty — and a very different effect on the coefficients.

$$\text{loss} = \text{MSE} + \alpha \cdot \sum |w_i|$$

## Ridge (L2)

Shrinks coefficients smoothly toward zero. Keeps every feature, just down-weighted.

## Lasso (L1)

Can push coefficients to exactly zero — it performs feature selection as a side effect.

# Feature Scaling Matters Here Too



Ridge and Lasso penalize coefficient size directly — so a feature's scale determines how harshly it gets penalized.

**Example:** temperature in °C (range ~20–80) next to velocity in m/s (range ~0–10). Without scaling, the regularizer squeezes the velocity coefficient far more than it needs to, just because velocity's raw values are smaller.

**Same Week 2 rule applies:** StandardScaler, fit on the training set only, inside a Pipeline — every time you regularize.

# Random Forest Regression



An ensemble of decision trees, each trained on a random subset of data and features. The forest's prediction is the average of all the trees' predictions.

## Strengths

- Captures nonlinearity with no feature engineering
- No scaling needed (it's tree-based)
- Hard to overfit as badly as a single deep tree

## Trade-offs

- Less interpretable than a linear model
- Can't extrapolate beyond the training range
- More compute than a linear fit

more details in [mlpi26\\_lect02\\_supplement3 slides and notebook](#)

# Choosing a Regressor

|                       | Linear/OLS | Ridge/Lasso     | Poly. Reg.          | Random Forest |
|-----------------------|------------|-----------------|---------------------|---------------|
| Interpretable?        | Yes        | Yes             | Somewhat            | No            |
| Needs scaling?        | No*        | Yes             | Depends             | No            |
| Handles nonlinearity? | No         | No              | Yes (risk: overfit) | Yes           |
| Overfitting risk?     | Low        | Low (by design) | High at high degree | Low-moderate  |

*\*Plain OLS doesn't strictly need scaling to fit correctly, but scaling still makes coefficients easier to compare.*

# Today's Lab: Calibrate, Overfit, Then Fix It

1

**Fit OLS on the calibration curve**

Single-feature voltage  $\rightarrow$  force; watch it underfit.

2

**Fit multiple linear regression**

Velocity + temperature + load  $\rightarrow$  friction torque.

3

**Sweep polynomial degree**

Watch train/test error diverge as degree grows.

4

**Apply Ridge & Lasso, then Random Forest**

Tame the overfit model; compare against a nonlinear ensemble.

# Key Takeaways



Regression predicts a number; evaluate it with RMSE and  $R^2$ , not accuracy.



More model flexibility isn't free — polynomial degree trades bias for variance.



Ridge and Lasso buy back overfitting risk; Lasso can also select features.

NEXT WEEK

## Ensemble Learning

Bagging, random forests, and gradient boosting — applied to predictive maintenance.